

프로그램 정의서: 차세대 AI 통합 보안 관제 플랫폼 (AI-SOC Platform)

1. 프로그램 개요 (Program Overview)

- 프로그램명 (가칭): SOC-Nexus (Security Operations Center Nexus)
- 목적: 이기종 SIEM(Sentinel, Splunk, Logpresso 등)에서 발생하는 대량의 보안 이벤트를 단일 접점에서 수집하고, 고객사별로 상이한 데이터 형식을 '노코드(No-Code)' 방식으로 정규화하여, 보안 분석가가 가장 효율적으로 업무를 처리할 수 있는 Jira 대체용 자체 티켓팅 시스템을 구축함.
- 핵심 가치:
 - 유연성 (Flexibility): SIEM 변경이나 고객사 추가 시 코드 수정 없이 UI에서 설정 가능.
 - 무결성 (Integrity): 하루 500건 이상의 이벤트 폭주 시에도 단 한 건의 누락 없는 처리 보장.
 - 지능화 (Intelligence): 단순 반복 업무를 제거하는 AI 자동 분석 및 초동 대응 지원.

2. 시스템 아키텍처 (System Architecture)

대량의 로그 유입에도 흔들리지 않는 **비동기 이벤트 기반 아키텍처(Event-Driven Architecture)**를 채택합니다.

2.1 논리 아키텍처 다이어그램 (Mermaid)

코드 스니펫

```
graph TD
    subgraph "External SIEM Sources"
        S1 -->|Webhook/LogicApp| GW
        S2 -->|Webhook Action| GW
        S3 -->|Stream Query| GW
        S4[Others] -->|REST API| GW
    end

    subgraph "SOC-Nexus Platform"
        GW[Universal Ingestion API] -->|Raw JSON| MQ
    end

    subgraph "Worker Cluster"
```

```
W1 -->|JSONPath Mapping| DB
W2 -->|LLM Prompting| DB
W3 -->|Email/Slack| Noti[Customer Contact]
end

MQ --> W1
W1 --> W2
W1 --> W3

DB

API <--> DB
UI[Frontend (Next.js)] <--> API
end
```

2.2 기술 스택 선정 (Tech Stack)

- **Frontend:** Next.js (React), Tailwind CSS (빠른 UI 개발), TanStack Query (실시간 데이터 동기화)
- **Backend:** Python FastAPI (고성능 비동기 API), Celery (백그라운드 작업 처리)
- **Database:** PostgreSQL (JSONB를 활용한 스키마리스 저장소, RLS 보안 적용)
- **Message Broker:** RabbitMQ (메시지 유실 방지 및 순서 보장, Dead Letter Queue 지원)
- **Cache:** Redis (중복 이벤트 제거 및 세션 관리)

3. 핵심 기능 상세 명세 (Core Features Specification)

3.1 유니버설 수집기 (Universal Ingestion Engine)

- 기능 정의: 어떤 형태의 JSON 데이터라도 오류 없이 받아내는 '만능 포수' 역할.
- 구현 로직:
 - POST /api/hooks/{uuid} 엔드포인트 하나로 모든 SIEM 요청을 수신.
 - 데이터 유효성 검사(Validation)를 최소화하고, 받은 즉시 **RabbitMQ**에 적재 후 200 OK 응답 (Latency < 50ms 목표).
 - **Zero-Data-Loss:** DB가 다운되어도 큐(Queue)에 데이터가 남아있어 복구 가능.

3.2 동적 필드 매핑 엔진 (Dynamic Field Mapping)

- 기능 정의: SIEM마다 다른 필드명(src_ip, SourceAddress, attackerIP)을 우리 시스템 표준 필드(src_ip)로 변환하는 로직을 UI에서 설정.
- 구현 방식: **JSONPath** 문법을 활용한 매핑 규칙 저장.
- UI 구성: "드래그 앤 드롭" 매퍼 제공 (왼쪽: SIEM 원본 로그 트리 -> 오른쪽: 우리 티켓 필드).

3.3 티켓 라이프사이클 관리 (Workflow Engine)

- 상태 머신(State Machine): 티켓의 상태 변화를 엄격하게 제어.
 - New → In Progress → Pending Customer → Resolved → Closed
- SLA 타이머: 심각도(High, Critical)에 따른 처리 제한 시간 카운트다운 및 임박 시 경고 알람.

3.4 AI 보안 분석가 (AI Copilot)

- 기능 정의: 수집된 로그를 바탕으로 AI가 1차 분석 리포트 자동 생성.
- 동작: 티켓 생성 즉시 AI Worker가 트리거되어, 공격 IP 조회(TI), 페이로드 분석, 대응 가이드라인을 작성하여 티켓의 'AI 의견' 란에 기입.

4. 데이터베이스 설계 (Database Schema Sketch)

유연성을 극대화하기 위해 정형 데이터와 비정형 데이터(JSONB)를 혼합하여 설계합니다.

4.1 ER 다이어그램 (Textual)

1. **tenants** (고객사 정보)
 - id (UUID), name, sla_policy (JSONB)
2. **alert_sources** (SIEM 연동 정보)
 - id, tenant_id, source_type (Splunk/Sentinel...), webhook_url_uuid
 - mapping_config (JSONB): 핵심! 필드 매핑 규칙이 저장됨.

JSON

```
{
  "title": "$.alert.name",
  "src_ip": "$.data.src_ip",
  "severity": "$.severity"
}
```

3. **tickets** (통합 티켓)
 - id (BigInt), tenant_id, status, severity, assignee_id
 - title, description (표준화된 정보)
 - raw_data (JSONB): SIEM에서 온 원본 로그 전체를 통째로 저장.
 - ai_analysis (Text): AI가 작성한 분석 요약.

5. UI/UX 디자인 스케치 (UI/UX Design)

5.1 메인 관제 대시보드 (Analyst Console)

3-Pane Layout을 적용하여 화면 이동 없이 모든 업무 처리 가능.

구역	화면 구성 요소	기능 설명
좌측 (List)	실시간 티켓 리스트	- 새로고침 없이 새 티켓이 위에서 아래로 흐름 (WebSocket) - 심각도별 색상 태그 (Red/Orange) - SLA 임박 시간 표시 (예: "15분 남음")
중앙 (Detail)	분석 워크스페이스	- AI 요약 카드: 최상단에 "이 공격은 SQL Injection으로 의심됩니다" 요약 노출 - 표준 필드: 정규화된 IP, 공격명 표시 - Raw Log: 접었다 폈다 할 수 있는 원본 로그 뷰어
우측 (Action)	도구 및 대응 패널	- Playbook 실행 버튼: "IP 차단", "고객 메일 발송" - 메일 미리보기: 고객에게 나갈 메일 초안 수정 - 과거 이력: 동일 IP/공격 유형의 과거 티켓 추천

5.2 관리자 설정 화면 (Admin Settings)

- 소스 추가 마법사:
 - SIEM 종류 선택 (Splunk, Sentinel 등)
 - 전용 Webhook URL 발급 및 복사 버튼
 - 샘플 로그 수신 대기: SIEM에서 테스트 이벤트를 쏘면 화면에 "수신 성공!" 표시와 함께 JSON 트리가 뜬.
 - 매핑 설정: 수신된 JSON 트리에서 필요한 값을 마우스로 끌어다 타겟 필드에 매핑.
-

6. 단계별 구현 계획 (Implementation Roadmap)

Phase 1: 핵심 골격 구축 (Core Foundation) - 4주

- 목표: 로그를 받고, 저장하고, 화면에 띄운다. (Jira 기본 기능 대체)
- 작업:
 - PostgreSQL DB 및 RabbitMQ 구축.
 - 범용 Webhook 수신 API 개발.
 - 기본적인 JSONPath 매핑 로직 구현 (하드코딩 설정).
 - 티켓 리스트 및 상세 조회 UI (Read-Only).

Phase 2: 동적 매핑 및 워크플로우 (Dynamic & Workflow) - 4주

- 목표: 개발자 없이 새로운 고객사를 연동한다.
- 작업:
 - 관리자용 "매핑 설정 UI" 개발 (드래그 앤 드롭).
 - 티켓 상태 변경 (State Machine) 로직 및 담당자 할당 기능.
 - 메일 발송 자동화 (Jinja2 템플릿 연동).

Phase 3: 지능화 및 고도화 (Intelligence & Scale) - 4주+

- 목표: 분석가의 시간을 절약한다.
- 작업:
 - OpenAI/Local LLM 연동하여 티켓 초동 분석 모듈 탑재.
 - 유사 티켓 검색 (Vector DB 활용).
 - 다크 모드 UI 및 단축키 지원 등 편의성 강화.

7. 결론

이 프로그램 정의서는 귀하가 구상하는 ****SIEM 연동형 AI 보안 관제 플랫폼****을 실현하기 위한 구체적인 지침서입니다. Jira의 한계를 넘어, 보안 관제에 특화된 유연하고 강력한 도구를 직접 구축함으로써 운영 효율성을 **200%** 이상 향상시킬 수 있을 것입니다.